NewsRadar: Mid-Term Progress Report

Deepjyoti Kakoti
Roll No.: 26M0774

Branch: `DEEPJYOTI`
Repository: `https://github.com/debord555/cs699_project`
Module: AI step (location extraction with a local language model)

1 Project Description

NewsRadar is a CS 699 (Software Lab) team project: a web application that scrapes news articles, extracts the places each article mentions, and plots them on an interactive map. It is built as a three-stage Python pipeline (scraping, location and topic mining, then a FastAPI backend with a React + Leaflet map) around a shared PostgreSQL + PostGIS database. The current version targets Indian news sources and geocodes places within India.

The main features of our application will be:

- News articles collected from multiple sources will be mapped to the locations they discuss, allowing users to explore what is happening across different regions of the world.
- News events from around the world will be represented as interactive points (blips) on a global map. Selecting a point will allow users to view the relevant news articles associated with that location.
- News websites will be continuously scraped to collect and update news data from multiple sources.
- Multiple articles reporting on the same real-world event will be identified and clustered together, preventing duplicate representation of the same event on the map.
- News will be categorized into different categories, including but not limited to Business, Sports, Military, Disasters, Crime, Environment, Technology, and Opportunities.
- Users will be able to filter news based on time and date ranges, allowing them to view recent or historical events.
- Users will be able to zoom into any geographical region and explore the news and events occurring in that area.
- The platform will provide links to the original news articles, allowing users to access the complete information from the respective news sources.
- Users will be able to search for news using text-based queries. The system will identify relevant news articles from across the world and display the corresponding events geographically on the map. For example, a search for "earthquake" would show relevant earthquake-related news from different locations around the world.

2 My part: the AI step

To put an article on the map, the system has to know *where* the story happens. Articles arrive as plain text, so a language model reads each one and lists the places it mentions, with their latitude and longitude. My work in this phase was to:

1. set up a language model that runs on our own computer, with one script; and
2. write the worker that sends each article to the model and stores the places it finds in the database.

The worker sits between the scraper, which saves articles, and the map, which reads places from the database.

3 Technology choices

Choice	What it is	Why it was chosen
Lemonade Server	A free program that runs language models on a local PC	No cost per request, no internet needed, and article text never leaves our machine. It offers the same interface (API) as OpenAI, so the code would work with a hosted model too.
Qwen3-8B (Q4_K_M)	An open language model with 8 billion parameters, compressed to about 5 GB	Small enough to run on a normal GPU, yet good at reading text and returning structured answers such as JSON.
Python	A programming language	Same language as the rest of the project; ready-made libraries for databases (`psycopg`) and web requests (`requests`).
PowerShell	The Windows command shell	Lets the whole model setup run as one script on our Windows machines.

4 Setting up the model

The script `scripts/setup_llm.ps1` prepares the model in one command:

```
powershell -ExecutionPolicy Bypass -File scripts\setup_llm.ps1
```

It runs these steps in order and stops with a clear message if one fails:

1. **Find Lemonade.** Looks for the `lemonade` program, first on the system path and then in its default install folder.
2. **Start the server.** If the server is not already running, starts it in the background and waits up to 60 seconds for its health check to pass.
3. **Download the model.** Downloads Qwen3-8B (about 5 GB) only if it is not already on the machine.
4. **Load the model** into memory.
5. **Smoke test.** Asks the model to reply with "OK" to confirm the whole chain works.
6. **Print the settings** (`LLM_BASE_URL`, `LLM_MODEL`) to copy into the private `.env` file.

The script is safe to run again: steps that are already done are skipped. The model name and port can be changed with the `-Model` and `-Port` options. I also added the two LLM settings to `.env.example`, so every teammate knows which values to set.

5 The location extractor

5.1 How it works

`llm/locate.py` runs as a loop that never stops:

1. Look for one article that is waiting to be processed: status `ready` and full text present. If there is none, wait 2 seconds and look again.
2. Send the article's text to the model and ask for the places it mentions.
3. For each place, check whether it is already in the `locations` table (ignoring upper/lower case). If not, add it with its coordinates.
4. Link the article to each place in the `article_location_map` table, keeping the order in which the places were mentioned.
5. Mark the article `llm_done`. If the model call failed, mark it `llm_failed` and save the error message instead.
6. Go back to step 1.

5.2 What the model is asked

The model is told to reply with *only* a JSON list, so the program can read the answer directly:

```
[{"name": "Guwahati", "latitude": 26.14, "longitude": 91.73},
 {"name": "Assam",    "latitude": 26.20, "longitude": 92.94}]
```

The temperature is set to 0, so the same article always gives the same answer. The prompt ends with `/no_think`, which turns off Qwen3's long "thinking" output and makes replies much faster.

5.3 Key design decisions

Decision	Reason
Use the article `status` column to find work	An article with no places would never get a row in `article_location_map`, so checking that table would pick the same article again forever. The status (`ready` → `llm_done` / `llm_failed`) marks every article as finished exactly once.
One database transaction per article	An article's places and its new status are saved together. If the program crashes halfway, nothing is saved and the article is simply tried again.
Lock the article while working on it (`FOR UPDATE SKIP LOCKED`)	Several copies of the worker can run side by side without two of them taking the same article.
Clean the model's answer before saving	Small models sometimes add extra text or repeat a place. The worker keeps only the JSON list, drops repeated names, and discards coordinates outside the valid range (latitude −90 to 90, longitude −180 to 180).
Reuse existing places	"Guwahati" and "guwahati" point to the same row, so the map shows one point per place rather than many.
Cut very long articles to 12,000 characters	Keeps each request within what the model can read at once.
Failed articles are marked, not retried forever	One bad article cannot block the queue, and the saved error explains what went wrong.

5.4 Settings

Setting	Value	Meaning
`LLM_BASE_URL`	`127.0.0.1:13305`	Address of the local model server
`LLM_MODEL`	Qwen3-8B	Model to use
`LLM_TIMEOUT`	300 seconds	Longest wait for one reply from the model
`LLM_MAX_BODY_CHARS`	12,000	Longest article text sent to the model
`LOCATE_IDLE_SECONDS`	2 seconds	Pause when there is no work

6 Description of the files

File	Purpose
`scripts/setup_llm.ps1`	Installs, starts, loads and tests the local language model with one command.
`.env.example`	Template settings; I added `LLM_BASE_URL` and `LLM_MODEL`.
`llm/locate.py`	The location extractor described in Section 5.
`llm/db.py`	Opens a connection to the database using the private `.env` settings.
`llm/settings.py`	Timeouts and limits for the extractor.
`llm/requirements.txt`	The three Python libraries the extractor needs, with exact versions.

To run the extractor:

```
cd llm
pip install -r requirements.txt
python locate.py
```

7 Outcome of this phase

- A local language model that any teammate can set up and test with one command.
- A working location extractor that turns article text into places with coordinates and links them to the article in the database.
- Safe for several workers at once, and never leaves half-saved results.
- All work is on GitHub (branch `DEEPJYOTI`).

8 Next steps

- Run the extractor on real articles from the scraper and measure how often the places and coordinates are correct.
- Extend the same step to tag each article with a news category.